Cognitive Robotics - Report

Chiara Benini

January 2026

1 Introduction

The goal of this project was to design and implement a cognitively inspired agent capable of navigating a simple grid world using the Nengo framework. The environment consists of walls and colour-coded locations (Red, Green, Blue, Yellow, and Magenta), and the agent is required to explore the environment, recognise colours despite sensory noise, and learn sequential relationships between coloured locations. In particular, the agent must detect and count colour transitions, focusing on how often different colours follow red during its navigation.

Nengo is grounded in the Neural Engineering Framework (NEF), which models cognition using populations of simulated neurons and dynamical systems principles. Accordingly, the agent is implemented using neural representations and transformations rather than symbolic rules, favouring biologically inspired mechanisms within a simplified simulation.

The agent receives three main types of sensory input: proximity sensors for wall detection, an RGB sensor encoding the colour of the current cell, and an RGB sensor encoding the colour of the next coloured cell in the direction of movement. These signals are continuous and noisy, requiring the agent to robustly classify colours and maintain internal representations over time. Based on these inputs, the agent implements a minimal form of memory that allows it to associate successive colour observations and accumulate statistics about colour transitions.

Overall, this report focuses on the agent's cognitive capabilities, namely perception, short-term contextual memory, and their interaction with behaviour, and on how these capabilities can be implemented using Nengo's neural abstractions. The following sections describe the design decisions made prior to implementation, the structure of the model, and an analysis of its behaviour and limitations.

2 Preliminary Choices

Before implementing the agent's functionality, several important design decisions were made regarding development tools, modelling frameworks, and the handling of sensory noise. These preliminary choices had a significant impact on the structure of the model and on the practicality of testing and extending it.

A first practical decision concerned the development environment. Although the model could be executed in standard Python environments, development was carried out primarily using the Nengo graphical user interface (GUI). The GUI provided a stable and interactive setting in which the agent's behaviour, neural activity, and internal signals could be observed in real time. While this choice limited the use of conventional debugging techniques such as step-by-step execution, it proved well suited to inspecting population-level dynamics and behavioural outcomes in a neural model.

A second key decision concerned the choice of modelling framework. Although Nengo provides higher-level cognitive abstractions through the Semantic Pointer Architecture (SPA), the implementation primarily relies on the Neural Engineering Framework (NEF). SPA is used

mainly as an organisational container for the overall model, while the core functionality is implemented directly through neural ensembles, connections, and functions. This choice reflects the sensorimotor nature of the task: continuous perceptual transformations, similarity evaluation, short-term memory, and movement control are more naturally and flexibly expressed using NEF mechanisms than through SPA's symbolic operations, which would be a more interesting choice for higher language-like dynamics and other similar structures. In this sense, SPA provides structural clarity, while NEF supplies the computational substrate that supports perception, memory, and action in a biologically inspired way.

Finally, several simplifying assumptions were introduced to keep the project manageable and aligned with the task specification. In particular, white cells are explicitly excluded from color classification and transition counting, as required. Additionally, while sensory noise was retained to preserve realism, noise levels were temporarily reduced during development to facilitate testing and debugging, allowing architectural issues to be isolated more easily before restoring the intended operating conditions.

3 Methods

3.1 Perception Layers

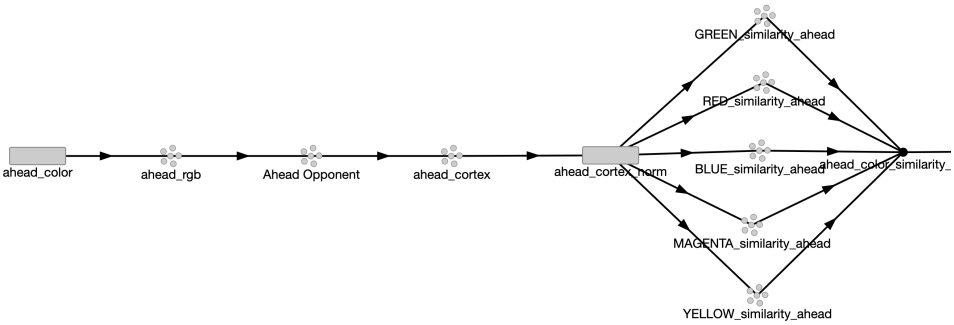

Figure 1: Overview of the perception pipeline implemented in Nengo, showing an example of one of the parallel processing for current and ahead colors through three hierarchical layers.

This section describes the perceptual strucutre employed by the agent, focusing on how raw colour information from the environment is transformed into a structured neural representation suitable for categorisation, comparison, and memory. The architecture is inspired by established models of biological colour vision (Kandel, Koester, Mack, & Siegelbaum, 2021), while remaining deliberately simplified to fit the constraints of the task and the Nengo enviroment.

3.2 Conceptual and Neuroscientific Motivation

The first task to tackle was particularly interesting since it gave an opportunity to implement a biologically inspired model. Looking at neuroscience resources, we can see that sight emerges from a layered process that starts with photoreceptors in the retina. There are two main types of photoreceptors: *rods* (for low-light vision) and *cones* (for color vision and bright-light vision). For this model, we focus primarily on cones since they are responsible for color vision.

Specifically, there are three kinds of cones, each tuned to a specific set of wavelengths:

- **L Cones (Long)**: Most sensitive to longer wavelengths (around 560-580 nm) and correspond broadly to what we can think as 'red receptors'. For simplicity, we have them correspond to the R channel in our RGB encoding, though the correspondence is not exact.

- **M Cones (Medium)**: Most sensitive to medium wavelengths (around 530-540 nm, yellow-green light)
- **S Cones (Short)**: Most sensitive to shorter wavelengths (around 420-440 nm, blue light)

The interesting aspect is how the brain and retina use these three elements to perform what we would call color categorization or classification. Signals from these cones pass through different layers and areas of the visual system, getting filtered and computed in specific ways. This is what we tried to imitate in a very simplified way through the Nengo implementation.

The perceptual architecture follows the same processing stages **in parallel for two visual inputs**: the color of the **current cell** occupied by the agent, and the colour of the **next non-white cell ahead**, if visible and not occluded by a wall.

3.3 Layer 1: RGB Input Encoding (Sensory Population Encoding)

The agent receives colour information as numerical RGB triplets derived from the grid-world environment. Each colour channel lies in the range $[0, 1]$, and Gaussian noise with standard deviation $\sigma = 0.01$ is added to simulate sensor uncertainty and test robustness. This noise value was chosen to be noticeable but not overwhelming, allowing the system to demonstrate robustness without causing complete failure of color discrimination.

Rather than processing these values numerically, each RGB signal is encoded into a **neural population** through the implementation of an ensemble. This conversion serves two purposes. First, it avoids non-neural "symbolic" processing by embedding perception directly in neural dynamics. Second, it introduces biologically plausible noise and variability, consistent with NEF principles.

White cells play a special role in the task and are not considered meaningful colours. To prevent noise from causing spurious classifications near white, nearly-white inputs are explicitly snapped to a fixed white reference when their Euclidean distance from pure white $[0.9, 0.9, 0.9]$ is less than 0.05. This threshold was chosen empirically to handle noise while avoiding excessive snapping that might interfere with legitimate color detection.

Conceptually, this stage, alongg with the initial perceptors of the agent, corresponds to **cone responses and early retinal processing** in the retina, where sensory signals are still closely tied to the physical stimulus.

3.4 Layer 2: Opponent Colour Transformation

This layer implements **opponent processing**, a strategy used by the visual system to make colors easier to distinguish. Instead of treating red, green, and blue independently, the brain compares them in pairs (asking questions like *"Is this more red or green?"* and *"Is this more blue or yellow?"*) while also separating brightness from color. Conceptually, this creates a new "color space" organized along opponent axes, where each dimension represents a meaningful contrast rather than a raw color channel. In this space, colors are represented by their position relative to these axes: for example, a strongly red color will lie far along the red–green axis, whereas a yellowish color will have a negative value along the blue–yellow axis. Compared to the previous RGB layer, this representation reduces redundancy and emphasizes perceptually relevant contrasts; compared to the next cortical layer, it is still low-dimensional and linear, serving as a structured foundation for more abstract and distributed representations.

After initial cone responses, signals are transformed into opponent channels in the **retina and lateral geniculate nucleus (LGN)** (Wikipedia contributors, 2024) . In human color vision, this involves :

1. **L-M (Red-Green) Channel**: Signals from L (red-sensitive) and M (green-sensitive) cones are compared, creating an opponent signal that signals either red (more L) or green (more M).

2. **S-(L+M) (Blue-Yellow) Channel**: Signals from S (blue-sensitive) cones are opposed by a combined signal from L and M cones, signaling either blue (more S) or yellow (more L+M).

3. **Luminance Channel**: A separate channel processes overall light intensity (brightness), often represented as (L+M).

Rather than using a direct LMS-to-opponent transformation, we implement a simplified linear transformation that captures the essential opponent relationships:

$$O_1 = R - G \quad \text{(red–green opponent)} \tag{1}$$

$$O_2 = B - \frac{R+G}{2} \quad \text{(blue–yellow opponent)} \tag{2}$$

$$O_3 = \frac{R+G}{2} \quad \text{(luminance)} \tag{3}$$

This transformation is implemented as a **fixed linear connection matrix** between the RGB ensembles and the opponent-colour ensembles.

3.5 Layer 3: Cortical Colour Space

The opponent signals are then projected into a "cortical" colour space represented by ensembles with **four dimensions**. The projection uses a simple identity transformation with an additional dimension that maps the 3D opponent space to 4D by carrying forward the three opponent dimensions unchanged and adding a fourth dimension initialized to zero. In biological visual cortex, representations become progressively higher-dimensional and more distributed as information flows from V1 to V2 to V4 (Conway, 2009). While the initial opponent representation is relatively low-dimensional (3D), cortical processing expands this into higher-dimensional spaces where colors are represented as points on complex manifolds. The fourth dimension was intended as a starting point for such expansion, with the expectation that future learning mechanisms or additional fixed transformations could populate this dimension with meaningful features. In the interest of time and project scope, these more advanced mechanisms were not implemented, leaving the fourth dimension at zero as a unused placeholder for potential future extensions.

A synaptic time constant $\tau = 0.01$ seconds is applied to this layer, introducing **temporal smoothing**, this is to stabilise the representation over time and mitigates the effects of sensory noise, which is particularly important for downstream memory and comparison mechanisms. The specific value was chosen to balance responsiveness (fast enough to track color changes during movement) and stability (slow enough to suppress high-frequency noise).

After transforming the values, color categorization then becomes a matter of finding the nearest prototype in this space, implemented as cosine similarity (Wikipedia contributors, 2025):

$$\text{similarity}(x, p) = x \cdot p = \|x\|\|p\| \cos\theta$$

Since both the cortical representation x and prototypes p are normalized to unit length, this reduces to $\cos\theta$, where θ is the angle between the vectors in the 4D space (3D plus placeholder).

The classification threshold was set to 0.4:

$$\text{class}(x) = \begin{cases} \text{color}_i & \text{if } \max_i \text{similarity}(x, p_i) \geq 0.4 \\ \text{white} & \text{otherwise} \end{cases}$$

This threshold was chosen to be high enough to reject spurious classifications due to noise, but low enough to allow legitimate color detections even with moderate noise.

Notably, the red color similarity ensemble was given additional neural resources (180 neurons vs. 150 for other colors) to improve its representation, as red transitions are central to the

task requirements. This allocation reflects a form of *attentional bias* toward task-relevant features, whiich can be also interepreted as biologically inspired (plasticity allocating more neural resources to detect more relevan stimuli).

In biological vision, color processing occurs hierarchically across visual areas: V1 extracts basic features like color contrasts, corresponding to our early encoding and opponent transformation; V2 integrates these features into more complex, distributed representations, similar to our cortical color space; and V4 encodes colors more abstractly and invariantly, approximated here by our similarity-based classification. While simplified, this model captures key principles of biological vision: hierarchical processing, distributed representations, graded similarity-based recognition, robustness to noise through temporal smoothing and population coding, and task-adaptive allocation of neural resources. Colors are thus represented as continuous, distributed patterns rather than discrete labels, enabling the agent to handle noisy inputs effectively and support downstream memory and decision-making.

3.6 Color Matching and Categorization

After transforming raw RGB signals through opponent processing and cortical representation, the agent must perform the critical task of **colour matching**, that is, mapping continuous perceptual representations onto a set of discrete colour categories (Green, Red, Blue, Magenta, Yellow). This section describes the normalization, similarity computation, and classification mechanisms used to implement categorical colour perception.

3.6.1 Normalization and Similarity Computation

The 4-dimensional cortical colour representations are **normalized to unit length** before being used for comparison. In practice, this means that each neural activity pattern is rescaled so that its overall strength is removed, while the relative activation across dimensions is preserved (SingleStore Documentation, 2025).

This choice is motivated by both neuroscientific intuition and NEF-based modelling considerations. Normalization makes the representation **independent of overall activation level**, which can vary due to noise, synaptic dynamics, or changes in luminance. As a result, colour information is primarily encoded in the *pattern* of activity across the neural population rather than in absolute firing rates.

Normalization also allows colour similarity to be computed using **cosine similarity**, which in this model reduces to a simple dot product between neural vectors (Luo, Zhan, Wang, & Yang, 2017). This provides a smooth and graded measure of perceptual similarity that is robust to noise and naturally compatible with Nengo's vector-based representations. Finally, keeping all representations at a fixed scale helps maintain stable neural dynamics and simplifies downstream comparison and decision processes.

3.7 Classification and Decision Threshold

Colour classification is implemented using a simple **winner-take-all mechanism with thresholding**. A custom node evaluates the similarity values for all five colour categories and selects the category with the highest similarity, provided that this value exceeds a fixed threshold. If no similarity exceeds the threshold, the input is treated as white (i.e., no colour).

The threshold value of 0.4 was determined empirically to balance two competing requirements:

- **Rejecting false positives**: avoiding classifications caused by noise or ambiguous inputs
- **Maintaining sensitivity**: allowing reliable colour detection even under moderate noise and variability

When a colour is identified, the system outputs a **one-hot-like vector** (for example, [0,1,0,0,0][0,1,0,0,0][0,1,0,0,0] for red), providing a discrete categorical representation suitable for downstream memory and decision-making mechanisms. This classification process is applied independently to both the current cell colour and the ahead cell colour, ensuring consistent handling of both perceptual streams.

3.7.1 Efficiency, Biological Plausibility, and Limitations

The proposed multi-stage colour processing pipeline represents a compromise between biologically inspired design and the specific requirements of the assignment task. Compared to simpler alternatives the model is quite robust to noise and variations in sensor readings, thanks to opponent processing and graded, population-based representations of colour similarity. Unlike a trained neural network classifier, the model is fully interpretable, does not require training data, and exhibits behaviour that can be directly linked to biologically motivated principles. Colours are encoded in distributed patterns of activity, and category decisions emerge through thresholding mechanisms, consistent with findings on human colour perception.

At the same time, the model necessarily simplifies biological reality. Processes such as long-term adaptation, synaptic plasticity, and colour constancy are omitted, and the dimensionality of the cortical representations is far lower than that found in real neural systems. Practical constraints also arise from the design choices: adding more colour categories or tracking all possible colour transitions would increase neural resource demands, reduce angular separation between representations, and heighten susceptibility to interference or misclassification. Extreme noise or inputs far from predefined prototypes can similarly destabilize classification, and spatial identity of colours is not explicitly represented. Within the scope of the assignment the system performs reliably, supporting both classification and transition counting tasks.

3.8 Red Transition Counting as Context Memory

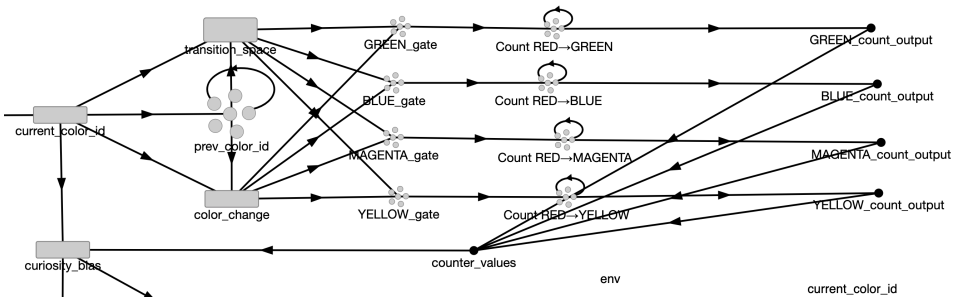

Figure 2: Red-transitions counting

The core cognitive requirement of this task is that the agent not only perceives colours but learns something about their sequential relationships; specifically, how often different colours follow red. This demands a form of memory. The agent must retain a trace of the previously visited colour and combine it with the current one to detect a transition event.

3.8.1 Maintaining a Trace of the Past: Leaky Memory

The first step is to maintain a record of the last colour encountered. This is implemented using a recurrent ensemble of 300 neurons representing a 5-dimensional vector (one dimension per colour category). Recurrent connections are configured with a transform of 0.9 and a synapse

of 0.1 seconds. This creates a classic *leaky integrator*: the ensemble's activity persists over time but gradually decays, holding onto the previous colour identity. A separate connection from the current colour classification updates this memory with a faster synapse of 0.05 seconds. The balance between the slow decay (90% retention per integration step) and the fast update ensures the memory is stable enough to avoid flickering between colours but responsive enough to update promptly when a genuine new colour is detected. Conceptually, this acts as a very short-term working memory for visual context.

3.8.2 Constructing a Transition Space

To detect a transition, the system must represent the *relationship* between the previous colour (P) and the current colour (C). A biologically plausible and algebra-inspired method is to use the outer product of their vector representations. Each color is represented as a "feature slot" , like a set of five switches for Green, Red, Blue, Magenta, and Yellow, where only the active color's switch is flipped. To capture a transition, the system asks: *"Given the previous color, which color is now active?"* This is implemented as a 5×5 grid where each row represents a previous color and each column represents a current color. A single cell lights up (is set to 1) only when that specific transition occurs, giving 25 possible transitions. Computationally, this is done by taking the **outer product** of the two 5-dimensional one-hot vectors and flattening the result into a 25-dimensional vector. Each index then uniquely corresponds to a specific color-to-color transition, making transitions like Red to Green easily identifiable. In Nengo, a custom node performs this by concatenating the previous and current vectors, computing their outer product, and outputting the flattened transition vector. This 25-dimensional representation serves as the explicit, structured foundation for detecting all possible color changes.

3.8.3 Gating and Selective Counting

A critical nuance is that a transition should only be registered when the agent moves from one coloured cell to a *different* coloured cell. Remaining on the same colour or moving within a white area should not trigger counting. To enforce this, a *colour-change detector* gates the counting mechanism. Another custom node compares the previous and current colour vectors. It outputs a signal of 1.0 only if a colour is currently detected and its identity differs from the stored previous colour. This signal acts as a gate: it is fed into a small ensemble along with the signal for a specific transition. Only when both the transition signal (e.g., "RedtoGreen is potentially happening") *and* the change signal ("a colour change is occurring") are present does a signal pass through to increment the corresponding counter.

As specified by the assignment, the agent only needs to count transitions originating from red. Therefore, counters are implemented only for indices in the 25D transition vector corresponding to RedtoGreen, RedtoBlue, RedtoMagenta, and RedtoYellow. Each counter is a single-dimensional ensemble with 200 neurons and a large radius (10), allowing it to represent potentially high counts. Crucially, each counter has strong recurrent self-connection with a transform of 0.99 and a synapse of 0.1 seconds. This configuration creates an integrator: any input pulse (a detected, gated transition) causes a near-permanent increase in the ensemble's represented value. The 99% retention factor means the count decays extremely slowly, creating a persistent statistical memory over the timescale of an agent's exploration.

3.8.4 Memory Characteristics and Handling Ambiguity

This counting mechanism forms the agent's primary episodic memory system. Its behaviour underlines a key difference between neural, statistical memory and symbolic, discrete memory. Consider an environment where the colour red appears in multiple locations, followed by green

in one context and blue in another. A symbolic system might try to store separate, context-bound rules ("at position X, red leads to green; at position Y, red leads to blue"). In contrast, this neural counter system does not discriminate contexts; it simply aggregates evidence. Both the Red to Green and Red to Blue counters will be incremented according to their respective frequencies. The memory does not resolve ambiguity, it reflects it.

This design choice has significant implications for the agent's behaviour and "understanding." The agent does not learn a deterministic map of transitions. Instead, it learns a statistical summary: "in my experience, red has been followed by green 60% of the time and by blue 40% of the time."

3.9 Integrating Curiosity with Wall Avoidance

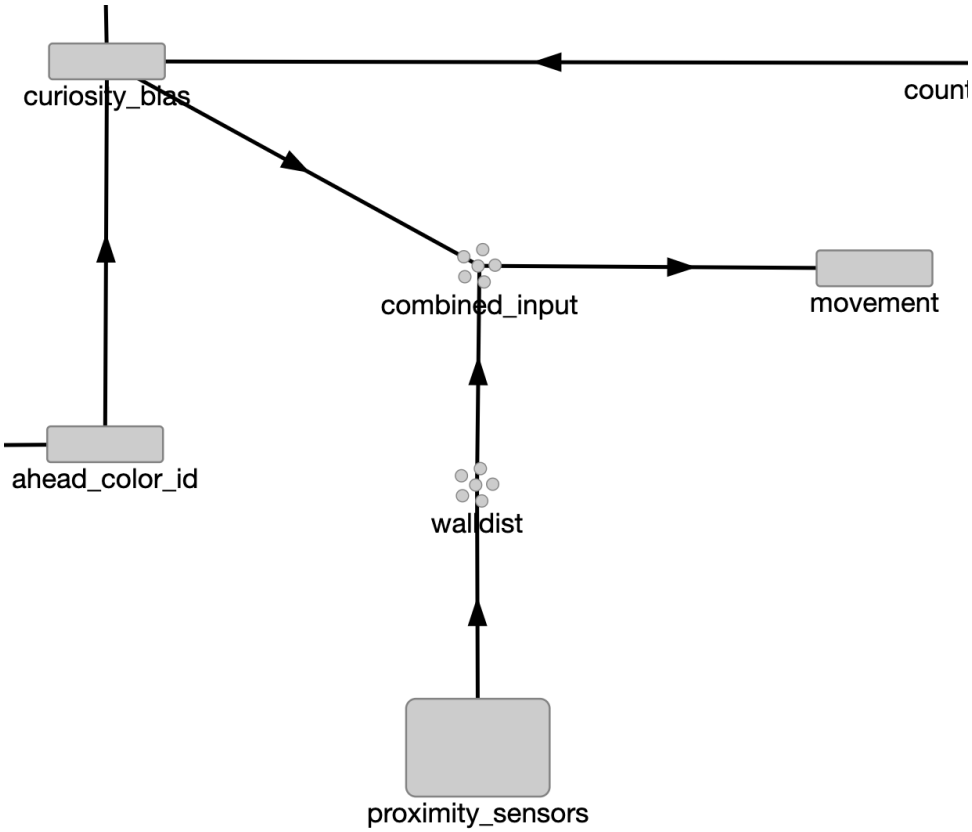

Figure 3: Interaction between movement, curiosity, and distance sensors

The base agent provided a competent wall-avoidance system using three proximity sensors. The primary challenge was augmenting this reactive navigation with a more cognitive, curiosity-driven behaviour without compromising safety. Rather than adding simple random perturbations, the design aimed implement some kind of behaviour loosely inspired by boredom avoidance or novelty-seeking: the agent should develop a preference for less-familiar paths based on its accumulated experience.

3.9.1 From Statistics to Steering Bias

The mechanism leverages the transition counters described in the previous section. When the agent finds itself on a red square (detected from the `current_color_id`), it examines the colour ahead (from `ahead_color_id`). If this ahead colour is one of the non-red colours being tracked (Green, Blue, Magenta, Yellow), the system calculates the *familiarity* of that specific Red to X

transition. Familiarity is defined as the proportion of all red-origin transitions that were of this particular type:

$$\text{familiarity} = \frac{\text{count}_{\text{Red}\to X}}{\sum_i \text{count}_{\text{Red}\to i}}$$

A high familiarity value (approaching 1.0) indicates that this transition has been experienced very frequently relative to others. To encourage exploration, this familiarity is converted into a steering bias. A scaling factor of -0.6 was determined empirically, producing the rule: bias $= -0.6 \times$ familiarity. The negative sign is crucial: it means the agent receives a turning command *away* from the direction that would lead to the overly familiar transition. This creates a soft repulsion from well-trodden paths.

The computation is encapsulated in a dedicated Nengo node (`curiosity_bias`). It takes three inputs: the current colour identity, the ahead colour identity, and the vector of the four transition counts. Its logic ensures the bias is only active when the agent is currently on red and facing a valid coloured square ahead, making the curiosity context-dependent.

3.9.2 Priority of Survival: Integrating with Wall Avoidance

The most significant implementation struggle was balancing this curiosity bias against the imperative of wall avoidance. A naive sum of the two commands lead the agent to drive into walls while being pushed around or slowed down excessively by both 'forces'. The solution was a **context-sensitive** gain system within the core movement function.

The movement function (`movement_func`) receives a 4D input: the three proximity sensor readings and the curiosity bias. It first handles emergency situations: if a wall is detected extremely close ahead (center sensor < 0.15), it executes a hard-coded evasive maneuver (sharp turn and slight reversal) completely overrriding any curiosity. For non-emergency navigation, it computes a primary turning command from the wall sensors (turn = right - left, steering away from closer walls).

The curiosity bias is then scaled based on how much "safe space" is available. This scaling uses the distance to the wall directly ahead as a proxy for safety:

- **Very close to a wall** (center < 0.3): curiosity is heavily attenuated (multiplied by 0.1). Survival is paramount.
- **Moderate space** ($0.3 \leq$ center < 0.6): curiosity has a moderate influence (multiplied by 0.7).
- **Clear path ahead** (center ≥ 0.6): curiosity exerts its full, strongest influence (multiplied by 1.5).

This scaled curiosity bias is then added to the wall-avoidance turn command. A final, very small amount of random noise is injected to break potential symmetrical deadlocks. The forward speed is also modulated by wall proximity, slowing down near obstacles.

3.9.3 Behavioural Implications and Tuning Challenges

The resulting behaviour is a continuous negotiation between two drives: a low-level, hard-wired reflex to avoid collisions, and a higher-level, experience-modulated tendency to explore novel transitions. In large, open environments, the curiosity mechanism becomes more visible, gently bending the agent's path away from frequently repeated Red toX sequences. In cluttered spaces, wall avoidance dominates, as intended but the complex behaviour becomes almost impossible to spot.

Tuning this system was iterative and non-trivial. The magnitude of the curiosity scaling factor (-0.6), the safety thresholds for scaling (0.3, 0.6), and the emergency threshold (0.15) were

all adjusted through trial and observation. Set too high, the curiosity bias could cause oscillatory "indecision" near walls or slow the agent to a crawl. Set too low, its effect became imperceptible. The final values represent a compromise where curiosity is a subtle but observable influence that modifies exploration over time without derailing the primary objective of safe navigation, but results might differ across maps and pre-set speeds. This architecture demonstrates a simple but effective form of behavioural integration, where a cognitive bias derived from learned statistics is seamlessly woven into a reactive control loop.

4 Expected Agent Behaviours

This section describes the qualitative behaviours expected from the agent based on its perceptual, memory, and control mechanisms. The agent was expected to navigate safely while reliably recognising colours under moderate noise, with white cells ignored and colour detections changing smoothly. Transition counters were expected to increment only for genuine red-origin colour changes, reflecting relative transition frequencies rather than exact histories. Movement would primarily follow wall avoidance, with curiosity gradually biasing exploration away from overly familiar red-origin transitions without overriding safety. Limitations included ambiguous statistics for repeated colours in different locations, occasional misclassifications under higher noise and across simlar colors, and potential slowdowns or oscillations when curiosity conflicted with collision avoidance.

5 Testing and Results

The agent was evaluated through interactive testing in the Nengo GUI across multiple grid-world configurations (also due to difficulties in alternative methods of testing, since VS code execution was avoided). Evaluation focused on qualitative behaviour, internal neural signals monitored thwough sliders and other devises, and the interaction between perception, memory, and movement, rather than on strict quantitative metrics.

5.0.1 Colour Recognition Performance

Under low to moderate noise levels, colour recognition was generally stable. When the agent entered a coloured cell, the corresponding category typically dominated the similarity computation, resulting in a clear classification. White regions were handled correctly in most cases, with the classifier remaining inactive. Some instability was observed for perceptually similar colours, particularly green and yellow, especially as noise increased, and when the agent was going too fast across single squares sometimes the reaction time for the colo recognitioin was lacking. These effects were consistent with overlap in the underlying representation space and confirmed that the system behaved as a graded perceptual model rather than a hard-coded classifier.

5.0.2 Transition Counting Behaviour

Red-origin transition counters increased over time in a manner broadly consistent with the agent's experienced trajectories. Frequently encountered transitions accumulated higher values, while rarely encountered ones remained low. Verifying exact correctness was challenging due to the continuous dynamics and limited debugging support in the GUI, and some missed or wrongful counts likely occurred under noisy conditions. Nevertheless, the overall mechanics seemed functional and the counters appeared to capture meaningful long-term statistics rather than arbitrary fluctuations.

5.0.3 Curiosity and Movement Dynamics

The curiosity mechanism produced a subtle influence on navigation. In larger, open environments, the agent's trajectories gradually diversified, showing avoidance of highly familiar transitions. In smaller or cluttered environments, wall avoidance dominated behaviour, and curiosity effects were often attenuated. Its also important to notice that the avoidance was not determined by sharp turns, so the behaviour is more akin to a slight drift ratther than the trajectories of wall avoidance. Occasional oscillations or slowdowns occurred when curiosity and wall avoidance generated competing commands. These effects were sensitive to parameter tuning and motivated conservative curiosity gains to preserve overall stability.

6 Discussion

This project highlights both the strengths and challenges of building cognitively inspired agents using Nengo. While the resulting behaviour was not always perfectly aligned with the intended design, the model provides useful insight into how perception, memory, and adaptive behaviour interact within a neural architecture.

A central challenge was the shift from conventional programming to reasoning about population-level dynamics. Unlike traditional systems, internal states in Nengo cannot be easily inspected or stepped through, making debugging and validation more difficult. This limitation was particularly apparent for the transition-counting mechanism, where confidence in correctness relied more on long-term consistency than on direct verification.

Despite these difficulties, the perceptual pipeline seems to be at least partially successful. The layered transformation from RGB input to similarity-based classification produced stable and noise-tolerant colour representations. Occasional confusions between similar colours were expected and are consistent with a graded, biologically inspired representation rather than a deterministic classifier.

The transition-counting mechanism functioned as a simple form of statistical memory. Rather than storing explicit episodes or spatial contexts, the agent aggregated experience over time. This made the system robust to variability but limited its ability to separate contexts or make precise predictions, so for future works a different approach might be necessary, one that would be more easily manipulated. Ambiguity was not resolved but instead reflected directly in the stored statistics.

The agent implements two distinct memory mechanisms: a short-term working memory for immediate context (enabling transition detection) and long-term statistical memory for accumulating transition frequencies. Although effective for the task, this design cannot separate transitions occurring in different spatial contexts; all red-to-green transitions are counted identically, regardless of location. From this starting point there would have been several interesting improvements that could've been considered for the memory implementation, given less constraints, for example, there are several algorithms for long-term memory, in particular using attractor physics (Hopfield Networks) that could've been adapted and integrated to render a more complex behaviour from the agent.

The curiosity mechanism represents the exploratory aspect of the design. By biasing behaviour away from familiar transitions, the agent exhibited a rudimentary form of adaptive exploration. However, this adaptiveness came at the cost of stability, particularly when curiosity conflicted with wall avoidance. These issues appear to stem primarily from behavioural interaction and parameter sensitivity rather than from a fundamental architectural flaw, but more sophisticated naviational strategies might be better suited to the task, especially if we aim to tackle more complex maps and behaviours.

Overall, the project illustrates a core tension in neural cognitive systems: increased biological plausibility and adaptiveness often reduce predictability and ease of control. Small parameter

changes can lead to large behavioural differences, making tuning both critical and challenging.

7 Conclusion

This project explored the design of a neurally inspired agent capable of perceiving colours, categorising them, and accumulating simple transition statistics within a grid-world environment. Using Nengo and population-based representations, the agent integrated sensory processing, memory, and control within a single continuous neural architecture.

The results demonstrate that relatively simple, biologically motivated mechanisms are sufficient to produce robust colour perception and meaningful experience-dependent behaviour under moderate noise. The agent seems to successfully distinguish the required colour categories, tracked red-origin transitions over time, and exhibited subtle exploratory biases driven by accumulated experience. At the same time, the project highlighted practical challenges inherent to neural modelling, including parameter sensitivity, limited interpretability, and trade-offs between robustness and flexibility.

Overall, this work shows how structured neural representations and principled transformations can support non-trivial cognitive behaviour without relying on training or symbolic reasoning. While the model remains a simplified abstraction of biological vision and learning, it provides a useful foundation for future extensions involving adaptive mechanisms, richer memory structures, or more complex environments.

References

Conway, B. R. (2009). Color vision, cones, and color-coding in the cortex. *Vision Research*, *49*(12), 1563–1574. doi: 10.1016/j.visres.2009.03.004

Kandel, E. R., Koester, J. D., Mack, S. H., & Siegelbaum, S. A. (2021). *Principles of neural science* (6th ed.). New York: McGraw-Hill Education. doi: 10.1036/007184141X

Luo, C., Zhan, J., Wang, L., & Yang, Q. (2017). Cosine normalization: Using cosine similarity instead of dot product in neural networks. (arXiv:1702.05870)

SingleStore Documentation. (2025). *Vector normalization.* `https://docs.singlestore.com/db/v7.5/reference/sql-reference/vector-functions/vector-normalization/`. (Accessed: 2026-01-22)

Wikipedia contributors. (2024). *Opponent-process theory.* `https://en.wikipedia.org/wiki/Opponent_process`. (Accessed: 2026-01-22)

Wikipedia contributors. (2025). *Cosine similarity.* `https://en.wikipedia.org/wiki/Cosine_similarity`. (Accessed: 2026-01-22)